PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

MemDB: In-Memory Key-Value Cache

Student / Team Information

Team Name: *Team #*	*Epoch*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Rawat, Arnav* *9068941214* *2510380192@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Mandal, Anish* *7596944379* *2510010615@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rawat, Suhani* *9520955384* *2510387319@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Semwal, Priyanshi* *9548428370* *2510011737@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Applications often access the same data repeatedly, even when that data changes infrequently. Retrieving such data repeatedly from a slower backing store can introduce unnecessary latency and additional backend operations. MemDB addresses this problem by providing a lightweight, application-level in-memory key-value cache for temporary and reconstructible data.
The project focuses on demonstrating how appropriate data structures and cache policies can improve repeated data access while operating within a bounded memory budget. MemDB supports key-based lookup, configurable capacity, TTL-based expiration, and multiple eviction policies.
The project also provides an opportunity to study the practical behavior of fundamental data structures under realistic cache operations and evaluate their effectiveness using measurable metrics such as cache hit rate, latency, eviction count, memory usage, and throughput.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Caching is commonly used as an application-level optimization to reduce repeated access to slower data sources. In-memory caches maintain frequently accessed data in RAM and use eviction policies such as LRU or FIFO when their capacity is reached. TTL mechanisms are also commonly used when cached data should remain valid only for a specified period.
MemDB implements these fundamental caching concepts as a compact embedded system. The core data structures remain within the application, providing explicit control over capacity, eviction, expiration, memory accounting, and statistics.
The project is intended primarily as an educational and experimental implementation that allows the behavior of the underlying algorithms and data structures to be visualized and benchmarked.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The primary goal is to implement a functional lightweight in-memory key-value cache while demonstrating fundamental data structures, algorithms, and object-oriented programming.

Initial milestones:

- Implement the hash table with collision handling using linked lists.
- Implement the doubly linked list required for LRU ordering.
- Implement a custom min-heap for TTL expiration management.
- Implement SET, GET, DELETE and CLEAR operations.
- Implement both LRU and FIFO eviction policies.
- Implement configurable memory capacity and memory usage tracking.
- Implement cache statistics including hits, misses and evictions.
- Build the C++ interface and policy abstraction using OOP.
- Develop a GUI for cache interaction and visualization.
- Implement benchmarking for progressively larger workloads.

The final milestone is an integrated demonstration showing insertion, lookup, collision handling, rehashing, LRU movement, eviction, TTL expiration, memory usage, statistics, and performance measurements.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

MemDB is designed as a layered system that separates the core data structures from cache management and the GUI.

The low-level data-structure layer is implemented in **C**. A hash table provides average $O(1)$ key-based lookup with linked-list collision handling. A doubly linked list maintains ordering information required by LRU eviction. A custom min-heap maintains expiration times so that the earliest expiring entry can be identified efficiently.

The higher-level cache management layer is implemented in **C++**. Classes encapsulate cache entries, cache management, statistics, memory accounting, and eviction policies. LRU and FIFO policies are represented through an object-oriented policy interface, allowing the eviction strategy to be selected without changing the core storage implementation.

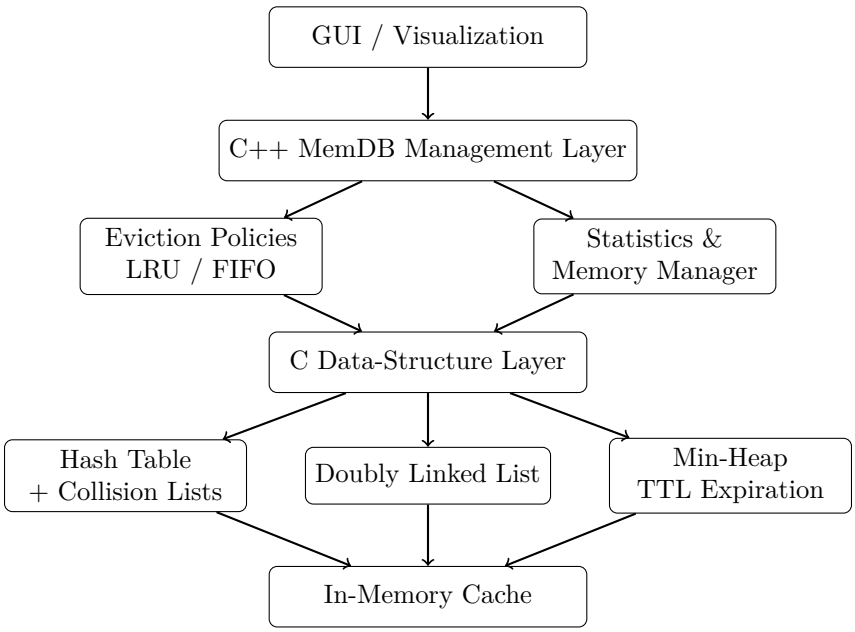
The cache maintains a configurable memory limit and tracks used and available memory. When capacity is exceeded, the selected eviction policy removes entries according to its rules. TTL expiration is managed using the expiration min-heap.

A GUI provides interactive visualization of cache contents, statistics, memory usage, the selected eviction policy, and the internal data structures. The system is intended to run as a desktop application using C and C++.

Finally, benchmarking will measure latency, throughput, cache hit rate, evictions, memory consumption, and performance as dataset size increases.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Operational flow:

GUI → C++ MemDB API → C Data Structures → In-Memory Cache

The C layer implements the core data structures, while the C++ layer manages cache policies, statistics, memory accounting, and interaction with the GUI.

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The primary deliverable is a working MemDB prototype implementing an in-memory key-value cache with configurable capacity, LRU and FIFO eviction policies, TTL-based expiration, memory tracking, and cache statistics.
The project will include the C implementation of the hash table, collision-handling lists, doubly linked list, and custom min-heap. The C++ layer will provide the object-oriented cache interface, eviction-policy abstraction, statistics management, and memory management.
A GUI will demonstrate cache contents, memory usage, statistics, selected eviction policy, and internal data-structure behavior. The system will provide interactive demonstrations of collision handling, rehashing, LRU movement, eviction, and TTL expiration.
Benchmark results will report performance for progressively larger workloads and compare the observed behavior with the expected algorithmic complexity.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words).

MemDB operates as an application-level cache for temporary or reconstructible data and does not replace persistent storage. The initial prototype operates primarily in memory within a single process. Persistence, networking, distributed synchronization, and multithreading are outside the core scope. Cache entries are assumed to have manageable key and value sizes, while the configured memory limit is assumed to be sufficient for the intended workloads. TTL values are based on the system clock.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Prentice Hall.
3. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.